



**UG Program in Electronics and Communication
(Advanced Communication Technology)**

Experiment No. 4

Error detection and correction using Cyclic Codes Encoding and Decoding.

4.1 Aim: Write python program for Error detection and correction using Cyclic Codes Encoding

4.2 Course Outcome: Compare the performances of different digital modulation techniques

4.3 Learning Objectives: Students will be able

- Understand the Fundamentals of Cyclic Codes

4.4 Related Theory:

A linear codeword is generally given as $c(n, k)$. Here n represents the total bits in the codeword and k denotes the message bits. Thus, the parity bits are $(n-k)$. Suppose we are given a code (7,4) then on comparing with general format the codeword will have 7 bits and the actual message bits are 4 while rest 3 are parity bits.

A cyclic codeword is given as:

$$C = [C_1, C_2, C_3 \text{ --- } C_{n-1}]$$

Then the codeword polynomial will be represented as:

$$C(X) = C_0 + C_1X + C_2X^2 + \text{---} + C_{n-1}X^{n-1}$$

For a given codeword $C = [1011]$, the codeword polynomial will be given as:

$$C(X) = 1 \cdot X^0 + 0 \cdot X^1 + 1 \cdot X^2 + 1 \cdot X^3$$

$$C(X) = X^3 + X^2 + 1$$

In a similar way, for any message codeword m , the message polynomial

$$M(X) = m_0 + m_1X + m_2X^2 + \text{---} + m_{k-1}X^{k-1}$$



UG Program in Electronics and Communication (Advanced Communication Technology)

And generator polynomial

$$G(X) = g_0 + g_1X + g_2X^2 + \dots + g_{n-k}X^{n-k}$$

Codewords are classified as systematic and non-systematic codewords.

A systematic codeword is one in which the parity bits and message bits are present in separated forms.

$C = [\text{parity bit, message bits}]$

But a non-systematic codeword is the one in which the message and parity bits exist in intermixed format and cannot be separated just by noticing the initial and final bits.

Let us now understand the coding and decoding of codewords using cyclic code.

Encoding

Non-Systematic Cyclic Encoding: Consider the message signal given as: $m = [1110]$

Thus,

$$M(X) = 1 \cdot X^0 + 1 \cdot X^1 + 1 \cdot X^2 + 0 \cdot X^3$$

$$M(X) = X^2 + X + 1$$

with generator polynomial $G(X) = X^3 + X + 1$

For non-systematic code, the codeword is given as:

$$C(X) = M(X) \cdot G(X)$$

$$C(X) = (X^2 + X + 1)(X^3 + X + 1)$$



**UG Program in Electronics and Communication
(Advanced Communication Technology)**

$$C(X) = X^5 + X^3 + X^2 + X^4 + X^2 + X + X^3 + X + 1$$

Here modulo 2 addition will be performed and in modulo 2 addition, Ex-or operation is performed

$$C(X) = X^5 + X^3 + X^2 + X^4 + X^2 + X + X^3 + X + 1$$

Thus, X^3 , X^2 and X will get cancelled. So, $C(X) = 1 + X^4 + X^5$

Hence, from the above codeword polynomial, the codeword will be: $C = [1000110]$

From the codeword bits, we can clearly interpret that the encoded codeword contains the message and parity bits in an intermixed pattern. Thus, is a non-systematic codeword and direct determination of message bits is not possible.

4.5 Program and Output:

Program:

```
import numpy as np

# 1. ENCODING (Non-Systematic Cyclic Code)

def encode_message(message, generator):

    m = np.poly1d(message)

    g = np.poly1d(generator)

    # Multiply message with generator polynomial

    codeword = np.polymul(m, g)
```



UG Program in Electronics and Communication (Advanced Communication Technology)

Modulo-2 operation on coefficients

```
codeword_mod2 = np.polyld(codeword.c % 2)
```

```
return codeword_mod2
```

2. SYNDROME CALCULATION

```
def calculate_syndrome(received, generator):
```

```
    r = np.polyld(received)
```

```
    g = np.polyld(generator)
```

```
    _, remainder = np.polydiv(r, g)
```

```
    syndrome = np.polyld(remainder.c % 2)
```

```
    return syndrome
```

3. ERROR DETECTION & CORRECTION

```
def correct_error(received, generator):
```

```
    n = len(received)
```

```
    syndrome = calculate_syndrome(received, generator)
```

```
    print("Syndrome:", syndrome)
```

```
    # If syndrome is zero → No error
```

```
    if np.all(syndrome.c == 0):
```

```
        print("No error detected.")
```

```
        return received
```

```
    print("Error detected! Trying to correct...")
```

```
    # Try single-bit error correction
```

```
    for i in range(n):
```

```
        test = received.copy()
```



UG Program in Electronics and Communication (Advanced Communication Technology)

```
# Flip bit (0→1 or 1→0)

test[i] = (test[i] + 1) % 2

test_syndrome = calculate_syndrome(test, generator)

if np.all(test_syndrome.c == 0):

    print(f'Error corrected at position {i}')

    return test

print("Unable to correct error.")

return received

# MAIN PROGRAM

# Message (example from experiment)

message = [1, 1, 1, 0]

# Generator polynomial  $G(x) = x^3 + x + 1$ 

generator = [1, 0, 1, 1]

# Encoding

encoded = encode_message(message, generator)

print("Encoded Codeword:", encoded)

# Convert polynomial to bit array

code_bits = list(encoded.c.astype(int))

# Introduce error manually

received = code_bits.copy()

received[3] = (received[3] + 1) % 2 # Flip one bit
```



UG Program in Electronics and Communication (Advanced Communication Technology)

```
print("Received Codeword (with error):", received)
```

```
# Detection & Correction
```

```
corrected = correct_error(received, generator)
```

```
print("Corrected Codeword:", corrected)
```

Output:

Encoded Codeword: $651x + 1x + 1x$

Received Codeword (with error): [1, 1, 0, 1, 0, 1, 0]

Syndrome: $1x + 1$

Error detected! Trying to correct...

Error corrected at position 3

Corrected Codeword: [1, 1, 0, 0, 0, 1, 0]

4.6 Conclusion:

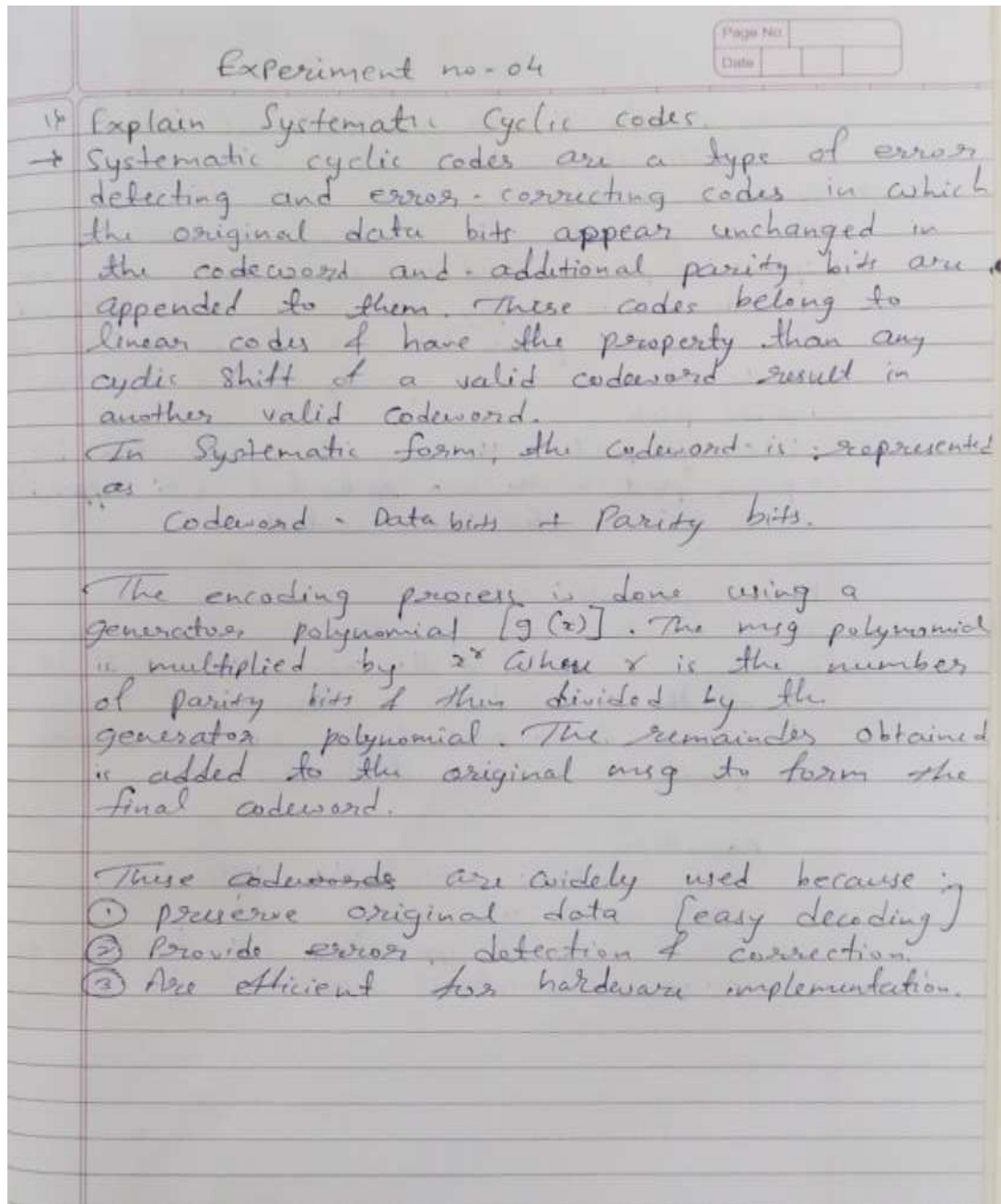
The experiment demonstrated encoding and decoding using cyclic codes for reliable data transmission. Errors were detected using the syndrome obtained from polynomial division. Single-bit errors were successfully corrected, proving the effectiveness of cyclic codes.



**UG Program in Electronics and Communication
(Advanced Communication Technology)**

4.7 Questions:

1. Explain Systematic cyclic codes





**UG Program in Electronics and Communication
(Advanced Communication Technology)**

2. What is Hamming distance, write formula for calculating Error correcting capability

Q. What is Hamming distance, write formula for calculating Error Correcting capability
→ Hamming distance is the number of positions at which the corresponding bits of two binary codewords are different

Example:

1011 and 1001 differ at 1 position, then Hamming distance = 1

∴ In simpler term: It measures how many bit errors are needed to change one codeword into another.

Formula:

d_{min} = minimum Hamming distance between any two codeword

• Error Detection Capability:

$$\text{Detectable errors} = d_{min} - 1$$

• Error Correction Capability:

$$t = \left\lfloor \frac{d_{min} - 1}{2} \right\rfloor$$

Where

t = number of errors that can be corrected

d_{min} = minimum Hamming Code.